REPUBLIC OF THE PHILIPPINES
BICOL UNIVERSITY
POLANGUI
COMPUTER STUDIES DEPARTMENT
Polangui, Albay

ISO 9001: 2015
SOCOTEC SCP000722Q

BAGONG PILIPINAS

IT112 Web System Development

Phase 6 – Update and Delete Functionalities

Lab Report by TechNet

Group Members:

Mico Malana

Christian Sannie Silo

Jay Lloyd Chicogo

John Nelson Andes

Allen Benedict Asuncion

Instructor:

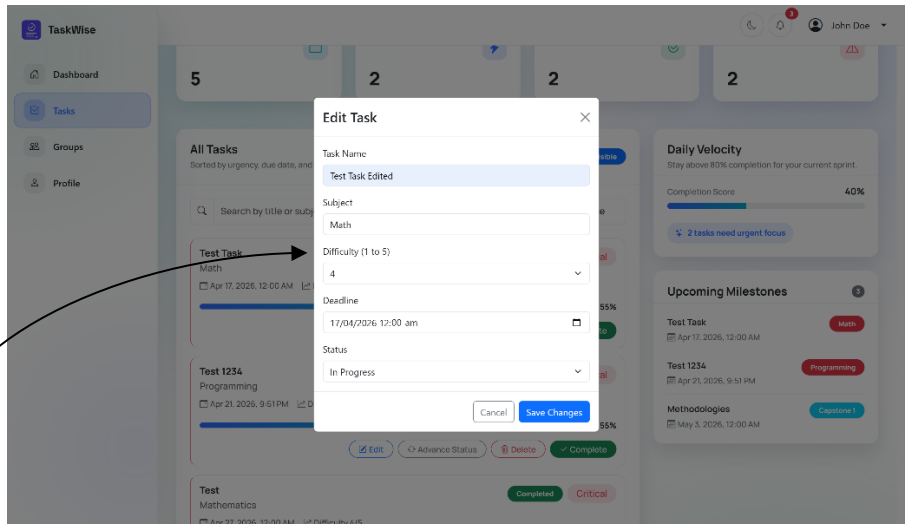
Guillermo V. Red, Jr., DIT

Note: We used *mongosh* for testing purposes to verify that the updated details were correctly saved in the collection, since the system is not yet in production.

I. Edit Modal/Page

The screenshot below shows the Edit Modal/Page for Tasks and Profile, where users are able to update task details and account information.

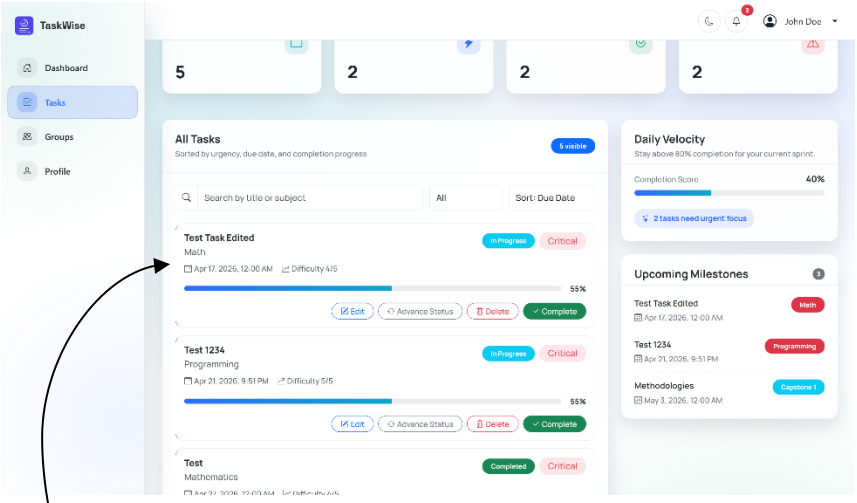
a. Edit Task Details



Here, it shows the Edit Task modal, where users can modify task details such as title, priority level, due date, and status.

Once the user saves the changes, the system processes the update and stores the new task details in the tasks collection:

```
{
  _id: ObjectId('69e05d8cf956ea67adfe6632'),
  name: 'Test Task Edited',
  difficulty: 4,
  deadline: ISODate('2026-04-16T16:00:00.000Z'),
  subject: 'Math',
  creator: ObjectId('69dce15740accb718f04916f'),
  createdAt: ISODate('2026-04-16T03:54:52.857Z'),
  updatedAt: ISODate('2026-05-01T12:06:01.046Z'),
  __v: 0,
  status: 'in-progress'
},
```



After a successful update, the system retrieves the latest task data and updates the state, causing the component to re-render. As a result, the updated task details are displayed instantly in the task list.

JavaScript Snippets:

Here, we destructure the useTasksData hook, which provides the editTask function used for updating tasks:

```
const {
  loading,
  error,
  stats,
  completion,
  upcomingMilestones,
  visibleTasks,
  query,
  setQuery,
  statusFilter,
  setStatusFilter,
  sortBy,
  setSortBy,
  completeTask,
  cycleTaskStatus,
  removeTask,
  addTask,
  editTask,
} = useTasksData(handleUnauthorized);
```

Here, we define the edit handler that sets the selected task's data and opens the Edit Modal with pre-filled values:

```
const handleOpenEdit = (task : any) : void => {
  setEditingTaskId(task.id);
  setEditValues({
    name: task.title || "",
    subject: task.subject || "",
    difficulty: Number(task.difficulty) || 3,
    deadline: toDateTimeLocalValue(task.deadlineISO),
    status: task.rawStatus || "pending",
  });
  setEditError(value: "");
  setShowEditModal(value: true);
};

const handleCloseEdit = () : void => {
  setShowEditModal(value: false);
  setEditingTaskId(value: null);
  setEditValues(initialForm);
  setEditError(value: "");
};
```

Here, we pass the handleOpenEdit function to the TaskList component, allowing each task item to trigger the Edit Modal when the edit action is selected:

```
<TaskList
  loading={loading}
  tasks={visibleTasks}
  query={query}
  onQueryChange={setQuery}
  statusFilter={statusFilter}
  onStatusFilterChange={setStatusFilter}
  sortBy={sortBy}
  onSortByChange={setSortBy}
  onEdit={handleOpenEdit}
  onComplete={completeTask}
  onCycleStatus={cycleTaskStatus}
  onDelete={handleOpenDelete}
/>
```

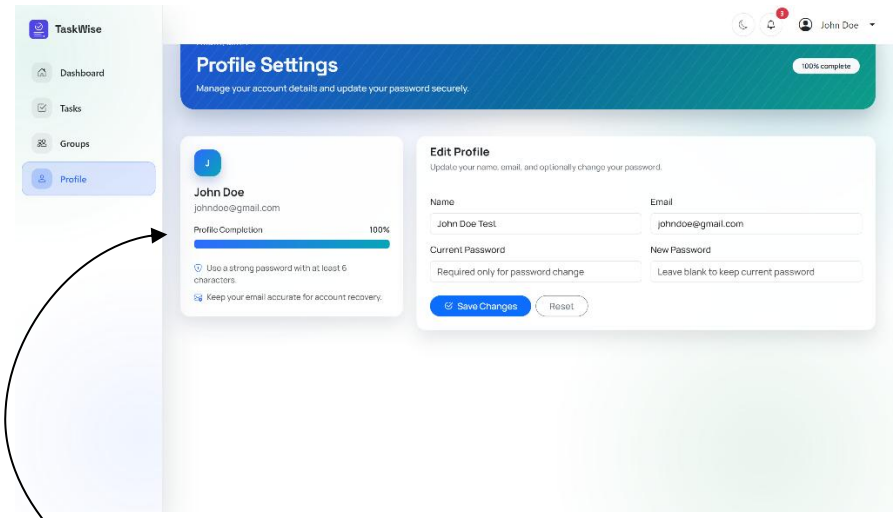
Here, inside TaskList, each task is rendered and the edit handler is passed so the selected task can be edited:

```
{tasks.map((task : any) : Element => (
  <TaskItem
    key={task.id}
    {...task}
    onEdit={() : any => onEdit(task)}
    onComplete={() : any => onComplete(task.id)}
    onCycleStatus={() : any => onCycleStatus(task.id, task.rawStatus)}
    onDelete={() : any => onDelete(task)}
  />
))}
```

Finally, we attach the edit handler to the TaskItem button:

```
<Button
  variant="outline-primary"
  size="sm"
  className="rounded-pill px-3"
  onClick={onEdit}
>
  <i className="bi bi-pencil-square me-1"></i>
  Edit
</Button>
```

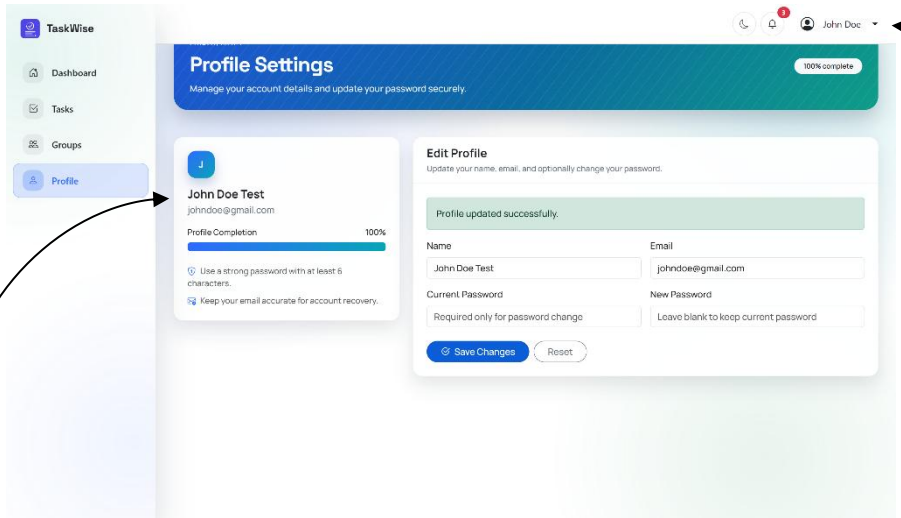
b. Edit Profile Details



Here, it shows the Profile Edit Page, where users can modify their account details such as name, email, and password.

Once the user submits the updated profile information, the system processes the request and stores the new details in the users collection:

```
{
  _id: ObjectId('69dce15740accb718f04916f'),
  name: 'John Doe Test',
  email: 'johndoe@gmail.com',
  password: '$2b$10$B/5s35prJYEZ4yrXjB3W4OiIA7qUJJ5C9Jq6H3S3oDYmWeJmeREIi',
  createdAt: ISODate('2026-04-13T12:28:07.968Z'),
  updatedAt: ISODate('2026-05-01T12:58:32.611Z'),
  __v: 0
},
```



After a successful update, the system refreshes the user data and re-renders the updated profile information in the interface.

The profile name displayed in the top bar requires a manual refresh to reflect the updated name.

JavaScript Snippets:

Here, we extract the saveProfile function from the useProfileData hook for updating profile details:

```
const {
  profile,
  formValues,
  loading,
  saving,
  error,
  saveError,
  successMessage,
  completion,
  updateField,
  resetForm,
  saveProfile,
} = useProfileData(handleUnauthorized);
```

Here, we define the submit handler and call saveProfile to update the profile:

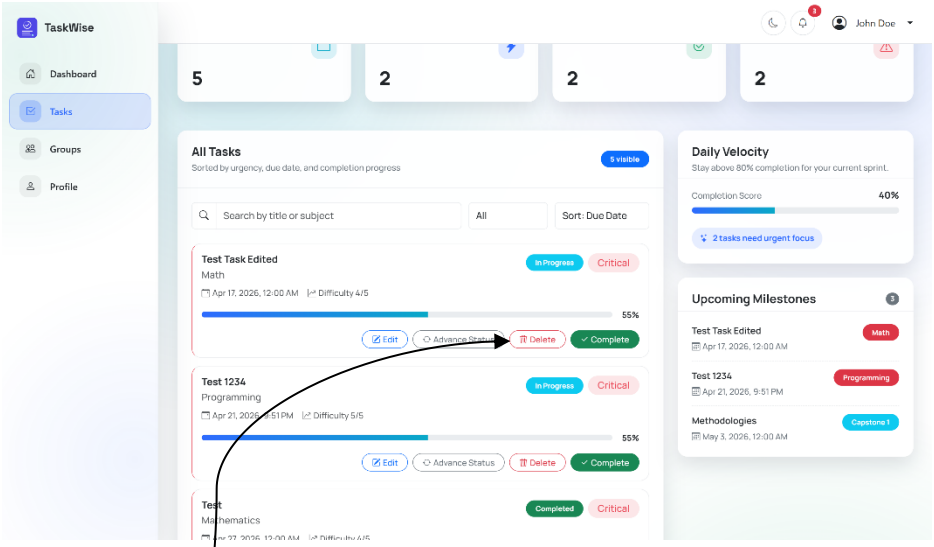
```
const handleSubmit = async (event : any) : Promise<void> => {
  event.preventDefault();
  await saveProfile();
};
```

Here, we define the profile edit form that handles user input and triggers handleSubmit when the user saves changes:

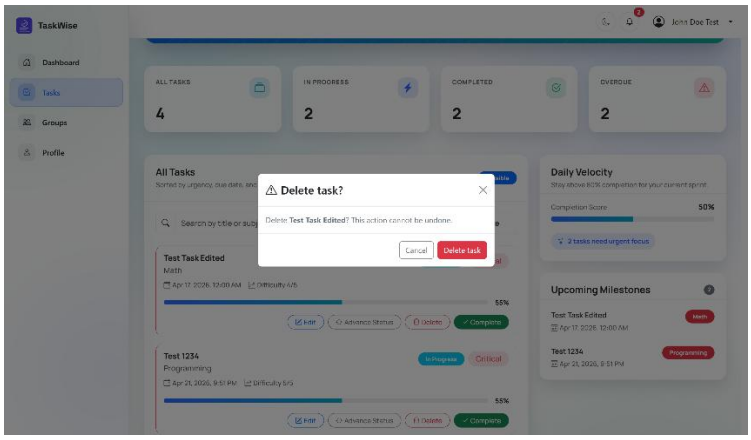
```
<Form onSubmit={handleSubmit}>
```

II. Deletion

a. Delete Task

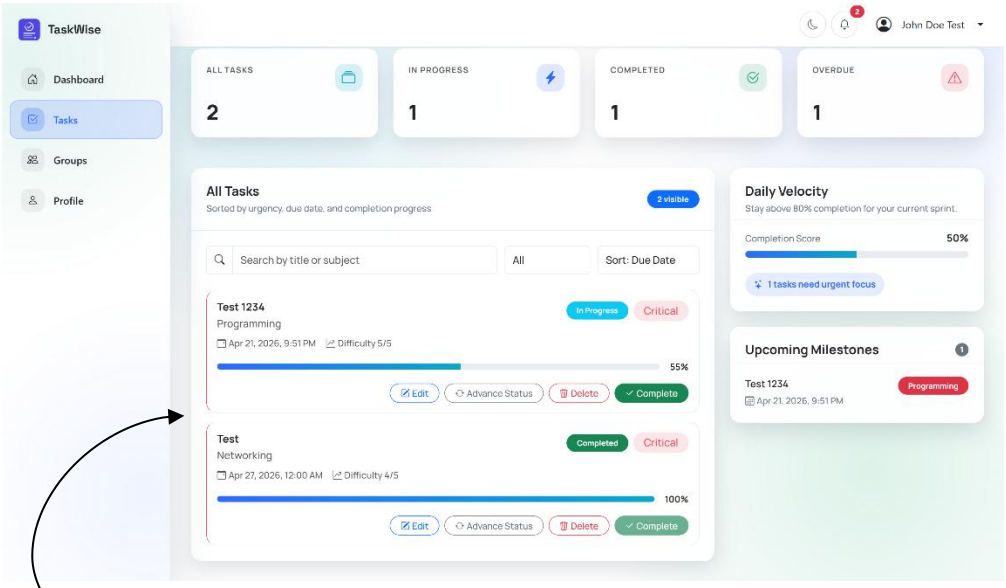


When a user attempts to delete a task, a confirmation modal is first displayed to prevent accidental deletion.



Once confirmed, the system processes the request and removes the task from the tasks collection:

```
studentDB> db.tasks.find()
[
  {
    _id: ObjectId('69e62f77bc388b1e0eafc687'),
    name: 'Test 1234',
    difficulty: 5,
    deadline: ISODate('2026-04-21T13:51:00.000Z'),
    subject: 'Programming',
    status: 'in-progress',
    creator: ObjectId('69dce15740accb718f04916f'),
    createdAt: ISODate('2026-04-20T13:51:51.601Z'),
    updatedAt: ISODate('2026-05-01T10:27:02.428Z'),
    __v: 0
  },
  {
    _id: ObjectId('69f18022ab715b8388f512c3'),
    name: 'Test',
    difficulty: 4,
    deadline: ISODate('2026-04-26T16:00:00.000Z'),
    subject: 'Networking',
    status: 'completed',
    creator: ObjectId('69dce15740accb718f04916f'),
    createdAt: ISODate('2026-04-29T03:50:58.854Z'),
    updatedAt: ISODate('2026-05-01T05:17:24.385Z'),
    __v: 0
  }
]
```



After successful deletion, the task list is updated and re-rendered to reflect the changes in the user interface.

JavaScript Snippets:

Here, we open the delete confirmation modal and store the selected task details:

```
const handleOpenDelete = (task : any) : void => {
  if (!task) {
    return;
  }

  setDeleteTarget({ id: task.id, title: task.title });
  setDeleteError(value: "");
  setShowDeleteModal(value: true);
};
```

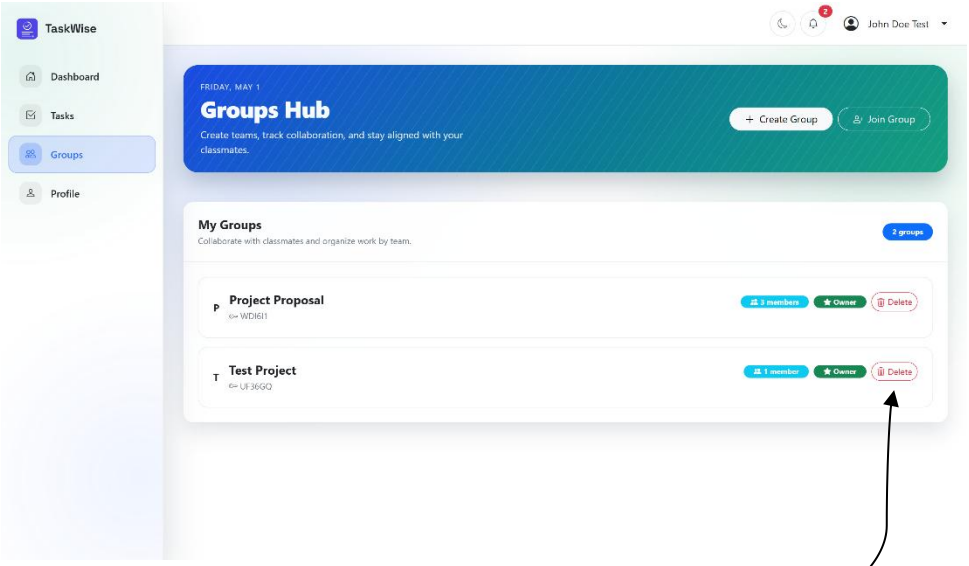
Then, we pass it to the TaskList component same as the Edit Task:

```
<TaskList
  loading={loading}
  tasks={visibleTasks}
  query={query}
  onQueryChange={setQuery}
  statusFilter={statusFilter}
  onStatusFilterChange={setStatusFilter}
  sortBy={sortBy}
  onSortByChange={setSortBy}
  onEdit={handleOpenEdit}
  onComplete={completeTask}
  onCycleStatus={cycleTaskStatus}
  onDelete={handleOpenDelete}
/>
```

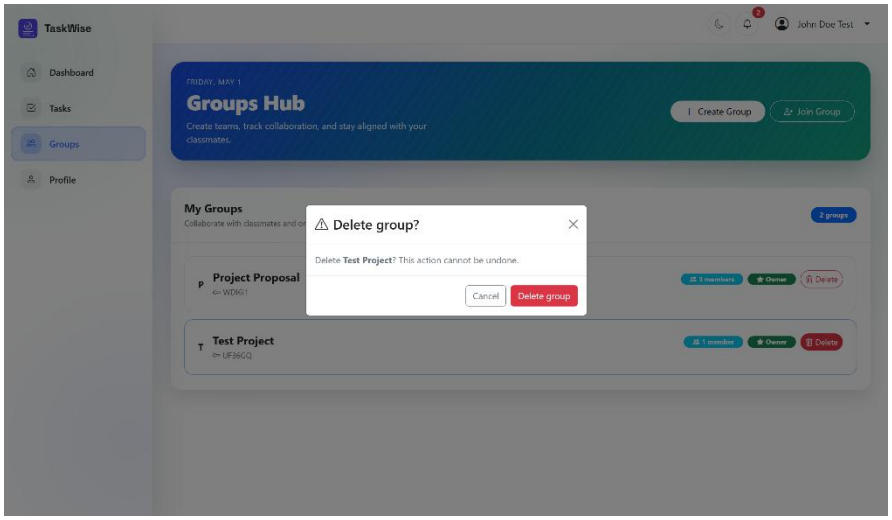
Then, we pass the handler down to the TaskItem button where it is attached and triggered:

```
<Button
  variant="outline-danger"
  size="sm"
  className="rounded-pill px-3"
  onClick={onDelete}
>
  <i className="bi bi-trash3 me-1"></i>
  Delete
</Button>
```

b. Delete Group

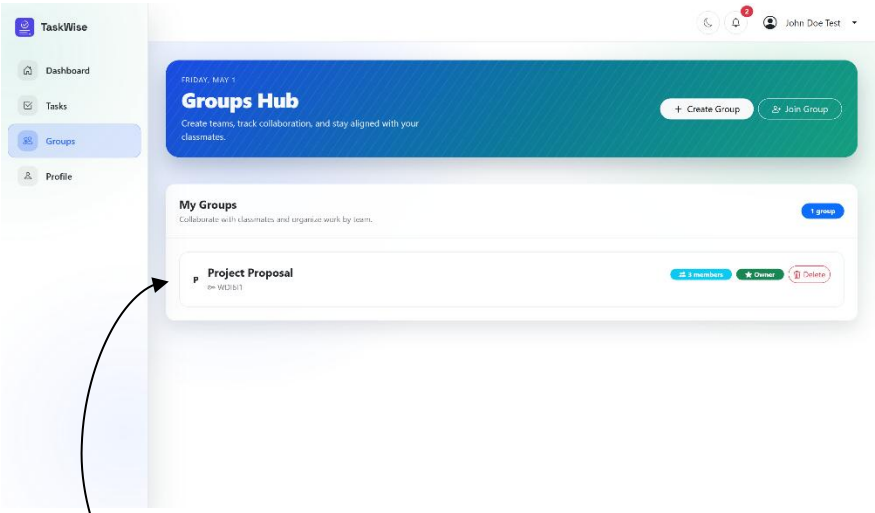


When a user deletes a group, a confirmation modal is first displayed to prevent accidental actions.



Once the user confirmed, the system then updates the groups collection accordingly:

```
studentDB> db.groups.find()
[
  {
    _id: ObjectId('69e646f085bf6e7ad48afa9a'),
    name: 'Project Proposal',
    owner: ObjectId('69dce15740accb718f04916f'),
    inviteCode: 'WDI6I1',
    createdAt: ISODate('2026-04-20T15:32:00.425Z'),
    updatedAt: ISODate('2026-04-20T15:32:00.425Z'),
    __v: 0
  }
]
```



After the action is completed, the group list is updated and re-rendered to reflect the changes in the user interface.

JavaScript Snippets:

Here, we destructure the useGroupsData hook which includes the deleteGroupById function used for deleting a group:

```
const {
  groups,
  loading,
  error,
  reload,
  groupDetails,
  detailsLoading,
  detailsError,
  assignableTasks,
  currentStudentId,
  addGroup,
  joinExistingGroup,
  deleteGroupById,
  leaveSelectedGroup,
  removeMemberFromGroup,
  assignTask,
  updateMemberTaskStatus,
} = useGroupsData(handleUnauthorized, selectedGroupId);
```

Here, we define the handler that opens the delete confirmation modal for a group and stores the selected group's details before deletion:

```
const handleOpenDelete = (group : any) : void => {
  if (!group) {
    return;
  }

  setDeleteTarget({ id: group.id, name: group.name });
  setDeleteError(value: "");
  setShowDeleteModal(value: true);
};
```

Here, we pass the delete handler to the GroupList component where it determines whether to open the delete modal for group owners or remove the user from the group if they are just a member:

```
<GroupList
  groups={groups}
  loading={loading}
  error={error}
  onReload={reload}
  onDelete={(group : any) : void | Promise<void> =>
    group?.isOwner ? handleOpenDelete(group) : handleLeaveGroup(group?.id)
  }
  selectedGroupId={selectedGroupId}
  onSelect={(groupId : any) : void | Promise<void> => navigate(to: `/groups/${groupId}`)}
/>
```

Finally, we attached it to every group item inside the GroupList component:

```
<Button
  variant="outline-danger"
  size="sm"
  className="rounded-pill"
  onClick={(event : MouseEvent<HTMLButtonElemen…) : void => {
    event.stopPropagation();
    onDelete?.(group);
  }}
>
  <i className="bi bi-trash3 me-1" />
  Delete
</Button>
```

III. Backend Route and Controller

a. Update Task

Route Layer

```
// UPDATE TASK
taskRouter.patch(
  path: "/task/update/:id",
  authMiddleware,
  validateRequest(validateUpdateTask),
  updateTask,
);
```

Controller Layer

```
const updateTask = asyncHandler(async (req : any, res : any) : Promise<void> => {
  const { id } = req.student;
  const newTask = await taskService.updateTask(req.params.id, id, req.body);
  res.status(200).json({ newTask });
});
```

Service Layer

```
const updateTask = async (taskId : any, studentId : any, taskInfo : any) : Promise<Document<unknown, {… => {
  const allowedFields = ["name", "difficulty", "deadline", "subject", "status"];
  const task = {};

  const existingTask = await findById(taskId);
  if (!existingTask) throw new NotFoundError(message: "Task not found");

  const isCreator = existingTask.creator.toString() === studentId.toString();
  let isAssigned = false;

  if (!isCreator) {
    isAssigned = await existsByTaskAndUser(taskId, studentId);
    if (!isAssigned) throw new ForbiddenError(message: "Unauthorized");
  }

  for (let key of allowedFields) {
    if (taskInfo[key] !== undefined) {
      if (!isCreator && key !== "status") {
        throw new ForbiddenError(message: "Assigned members can only update task status");
      }
      task[key] = taskInfo[key];
    }
  }

  const newTask = await updateById(taskId, task);

  if (!newTask) {
    throw new NotFoundError(message: "Task not found");
  }

  return newTask;
};
```

Repository Layer

```
const updateById = async (taskId : any, task : any) : Promise<(Document<unknown, … => {
  return await Task.findByIdAndUpdate(
    taskId,
    { $set: task },
    { new: true, runValidators: true },
  );
};
```

b. Delete Task

Route Layer

```
// UPDATE TASK
taskRouter.patch(
  path: "/task/update/:id",
  authMiddleware,
  validateRequest(validateUpdateTask),
  updateTask,
);
```

Controller Layer

```
const deleteTask = asyncHandler(async (req : any, res : any) : Promise<void> => {
  const { id } = req.student;
  const task = await taskService.deleteTask(req.params.id, id);
  res.status(200).json({ task });
});
```

Service Layer

```
const deleteTask = async (taskId : any, studentId : any) : Promise<Document<unknown, {… => {
  const task = await findByIdAndCreator(taskId, studentId);
  if (!task) throw new ForbiddenError(message: "Unauthorized");

  await deleteById(taskId);
  return task;
};
```

Repository Layer

```
const deleteById = async (taskId : any) : Promise<(Document<unknown, … => {
  return await Task.findByIdAndDelete(taskId);
};
```

c. Delete Group

Route Layer

```
// DELETE GROUP
groupRouter.delete(path: "/group/delete/:id", authMiddleware, deleteGroup);
```

Controller Layer

```
const deleteGroup = asyncHandler(async (req : any, res : any) : Promise<void> => {
  const { id } = req.student;
  const group = await groupService.deleteById(req.params.id, id);

  res.status(200).json({ group });
});
```

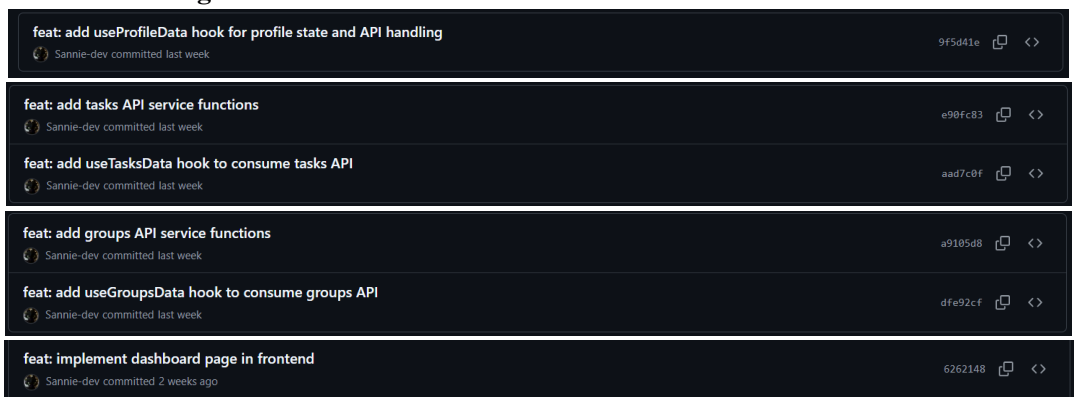
Service Layer

```
const deleteById = async (groupId : any, studentId : any) : Promise<Document<unknown, {… => {
  const group = await groupRepo.findByGroupId(groupId);

  if (!group) throw new NotFoundError(message: "Group not found");

  if (String(group.owner) !== String(studentId)) {
    throw new ForbiddenError(message: "Only admin can delete group");
  }

  await Promise.all([
    deleteAssignmentsByGroupId(groupId),
    deleteByGroupId(groupId),
    groupRepo.deleteById(groupId),
  ]);

  return group;
};
```

Repository Layer

```
const deleteById = async (id : any) : Promise<(Document<unknown, … => {
  return await Group.findByIdAndDelete(id);
};
```

IV. GitHub Logs



V. Member Reflections

- **Mico Malana (Project Manager, Documentation Officer)**
  As the Project Manager, my job is to monitor and to make sure that each changes aligns with our project and to also give instructions to my groupmates. Also as the Document Officer my job was to prepare screenshots and its explanations for our lab report.

- **Christian Sannie Silo (Frontend Developer)**
  In this phase, I was assigned to connect the update and delete functionalities to the backend. I made sure that user actions are correctly processed and reflected in the system. This helped me understand better how frontend and backend works together.

- **Jay Lloyd Chicogo (GitHub Manager, Backend Developer)**
  I supported the frontend in connecting the update and delete functionalities to the backend. I made sure the flow of the backend is clear so the frontend knows how the data is processed. This helped improve our understanding of how the system works together.

- **John Nelson Andes (Database Manager)**
  I was assigned to ensure that the data in the collections is successfully updated or deleted. I verified that all changes are properly reflected in the database. This helped maintain data consistency in the system.

- **Allen Benedict Asuncion (Tester & Debugger)**
  I was assigned to test the update and delete functionalities to ensure they respond correctly. I verified that the system properly renders the updated data after each action. This helped ensure that the features are working as expected.